



OSR eHSM OTP 数据生成工具用户使用指南

版本	1.3
作者	OSR IP
部门	OSR IP
日期	2026 - 08 - 11
分类	用户手册

Copyright Notice and Proprietary Information Notice

Copyright © 2014 - 2026 Open Security Research, Inc. All rights reserved. This documentation contains confidential and proprietary information that is the property of Open Security Research, Inc. The documentation is transported under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, translated, distributed, disclosed, announced or copied in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Open Security Research, Inc., or as expressly provided by the license agreement.

版权声明和专有信息声明

版权所有 © 2014 - 2026 深圳市纽创信安科技开发有限公司 保留所有权利。本文档包含机密和专有信息，属于深圳市纽创信安科技开发有限公司的财产。该文档根据对应的许可协议传播，且只能根据许可协议的条款使用或复制。未经深圳市纽创信安科技开发有限公司事先书面许可或经许可协议明确规定，该文档和对应软件的全部或任何部分均不得以任何形式或方式（电子的，机械的，手动的，光学的或其他的）被复制、传播、翻译、分发、披露、宣布或抄袭。

修订历史

版本	修改日期	描述
1.0	2025-07-04	初始版本
1.1	2025-10-28	更新排版、标题、字体等配置，增加 rtl_key 的字节序说明
1.2	2026-01-22	新增密钥文件格式章节，支持通过 key_file 引用外部密钥文件
1.3	2026-02-11	gen 命令新增 --c-array 选项，支持输出 C 数组源文件；C 库新增 gen_c_array 函数

目录

1 简介	5
2 Layout 文件格式	6
2.1 <code>[[config]]</code> - 全局配置	6
2.2 <code>[[bytes]]</code> - 字节对齐字段	6
2.3 <code>[[bits]]</code> - 比特位字段	7
2.4 <code>[[key]]</code> - 密钥字段	7
3 Values 文件格式	9
4 密钥文件格式	11
4.1 支持的密钥类型	11
4.2 对称密钥文件格式	11
4.3 非对称密钥文件格式	11
4.4 密钥提取规则	12
5 otptool 命令行工具使用	13
5.1 <code>gen</code> - 生成 OTP 数据	13
5.2 <code>parse</code> - 解析 OTP 数据	13
5.3 <code>show-layout</code> - 可视化布局	13
6 C 库使用	15
6.1 <code>gen_c_array</code> - 生成 C 数组源文件	15
7 示例	16
8 常见问题	18

1 简介

OSR OTP Tool 是一套功能强大的开发工具，旨在帮助嵌入式系统开发者高效、准确地生成和解析 OTP (One-Time Programmable) 内存数据。OTP 内存一旦编程，其内容便无法更改，因此在生产前确保数据正确至关重要。本工具通过结构化的配置文件，将复杂的内存位域操作转化为易于管理和审计的文本文件，极大地降低了出错风险。

本工具提供两种核心使用方式，以适应不同的开发流程：

1. `otptool` 命令行工具: 一个独立的、跨平台的可执行文件，用于在开发和生产阶段快速生成二进制 OTP 数据文件或解析已有的二进制文件进行验证。
2. C 静态链接库 (`otptool_clib`): 一个标准的 `.a` 静态库和对应的 `.h` 头文件，可以轻松集成到现有的 C/C++ 项目或生产工具链中，实现 OTP 数据的程序化生成与解析。

2 Layout 文件格式

可参考 `toml-demo/layout.toml` 文件。

Layout 文件是整个工具的核心，它使用 TOML 格式定义了 OTP 内存的完整布局、每个数据字段的位置、长度和类型。一个典型的 Layout 文件包含以下几个部分：

2.1 `[config]` - 全局配置

此部分定义了 OTP 数据的全局属性。

- `otp_size` (整数, 必需): OTP 内存的总大小, 单位为字节。
- `default_one` (布尔值, 必需): 定义 OTP 内存区域的默认填充值。如果为 `false`, 未定义区域的比特位默认为 `0`; 如果为 `true`, 则默认为 `1`。
- `rtl_key` (字符串, 可选): 根密钥, 通常用于加密其他密钥。它是一个 16 字节或 32 字节的十六进制字符串。

示例:

```
1| [config]
2| # OTP 共有 512 字节
3| otp_size = 512
4| # OTP 的默认值是 0
5| default_one = false
6| # RTL KEY 值
7| rtl_key = "efcdab89674523012064a8ec3175b9fd"
```

注意: 配置文件中的 `rtl_key` 密钥值是按照行业惯例的字节顺序进行排列, 但这与 eHSM 的 RTL 代码中定义的密钥值的字节顺是相反的, 请注意区别。例如上述示例中的密钥值在 RTL 代码中定义如下:

```
1| `define OSR_RTL_KEY    128'hFDB97531_ECA86420_01234567_89ABCDEF
```

2.2 `[[bytes]]` - 字节对齐字段

用于定义以字节为单位的字段。

注意, 字节序为小端。

- `name` (字符串, 必需): 字段的唯一名称。
- `offset` (整数, 必需): 字段的起始偏移地址, 单位为字节。
- `length` (整数, 必需): 字段的长度, 单位为字节。
- `add_crc32` (布尔值, 可选): 如果为 `true`, 工具会在该字段数据的末尾自动追加一个 4 字节的 CRC32 校验和。此时, 该字段的总长度会变为 `length + 4` 字节。
- `choices` (哈希表, 可选): 提供一组预设的具名选项。在 `values.toml` 中可以直接使用选项的名称, 而无需填写冗长的十六进制字符串。

示例:

```
1| [[bytes]]
2| name = "uid"
3| offset = 4
4| length = 16
5| add_crc32 = true # 最终数据会是 16 字节 UID + 4 字节 CRC
6|
7| [[bytes]]
8| name = "lifecycle"
9| offset = 0
10| length = 4
11| # 为生命周期状态提供具名选项
12| choices = { test = "00000000", dev = "14848142", user = "96ecc157" }
```

2.3 [[bits]] - 比特位字段

用于定义不对齐到字节的、更精细的比特位字段。

注意, 位序为小端。

- `name` (字符串, 必需): 字段的唯一名称。
- `offset` (整数, 必需): 字段所在字节的偏移地址。
- `bit_offset` (整数, 必需): 字段在 `offset` 字节后的起始比特位偏移量。
- `width` (整数, 必需): 字段的位宽度, 单位为比特。
- `choices` (哈希表, 可选): 提供一组预设的具名选项, 值为比特位字符串 (小端) 或整数值。

示例:

```
1| [[bits]]
2| name = "bus_cipher_en"
3| offset = 24 # 在第 24 字节
4| bit_offset = 0 # 从第 0 位开始
5| width = 2 # 宽度为 2 比特
6| choices = { enable = "00", disable = "10" }
```

2.4 [[key]] - 密钥字段

专门用于定义密钥结构。密钥字段固定占用 40 字节。

- `name` (字符串, 必需): 密钥的唯一名称。
- `offset` (整数, 必需): 密钥字段的起始偏移地址, 单位为字节。

示例:

```
1| [[key]]  
2| name = "chip_root_key"  
3| offset = 0x70
```

注意：受 eHSM 密钥方案要求，如果在 layout.toml 中定义了 `[[key]]` 字段，则必须在 `[config]` 中定义 `rtl_key` 的值，必须在 `[[bits]]` 中定义 `key_alg_sel` 字段，且前两个 `[[key]]` 字段的 `name` 必须为 `chip_root_key` 和 `device_root_key`。

3 Values 文件格式

可参考 `toml-demo/values.toml` 文件。

Values 文件同样使用 TOML 格式，它为 Layout 文件中定义的每个字段提供具体的值。

- **键名:** 必须与 Layout 文件中字段的 `name` 完全匹配。
- **值:**
 - ▶ 对于 `bytes` 字段，值通常是一个十六进制字符串，或 `choices` 中定义的名称。
 - ▶ 对于 `bits` 字段，值可以是二进制字符串 (如 `"1011"`)、`choices` 中定义的名称，或者一个整数。当提供整数时，工具会根据字段的 `width` 自动将其转换为正确长度的二进制表示。
 - ▶ 对于 `key` 字段，值是一个 TOML 表，包含密钥的详细属性。

密钥 (`key`) 字段的详细结构:

- `level` (整数): 密钥等级 (1 或 2)。
- `lifecycle` (字符串): 密钥生命周期 (`unburned`, `unused`, `available`, `disabled`, `destroyed`)。
- `type` (字符串): 密钥类型 (`symm`, `priv`, `pub`)。
- `value` (字符串): 密钥的十六进制值。当 `hash` 为 `false` 时，该值必须为 32 字节 (64 个十六进制字符); 如果密钥本身不足 32 字节，需要手动填充。与 `key_file` 互斥。
- `key_file` (字符串, 可选): 密钥文件的路径，支持相对路径 (相对于当前 `values.toml` 文件所在目录) 或绝对路径。密钥文件格式请参考下文的“密钥文件格式”章节。与 `value` 互斥。
- `hash` (布尔值或字符串, 可选): 控制是否在加密前对 `value` 进行哈希运算。
 - ▶ 如果为 `false`、`"none"` 或不提供该字段，则不进行哈希。
 - ▶ 如果为 `true` 或 `"auto"`，将根据 `key_alg_sel` 字段自动选择哈希算法 (SM4 对应 SM3, AES 对应 SHA256)。
 - ▶ 可以显式指定为 `"sm3"` 或 `"sha256"` 来强制使用特定的哈希算法。
- `last_key` (布尔值, 可选): 标记是否为最后一个密钥。
- `no_crc32` (布尔值, 可选): 标记是否禁止生成 KEY 的 CRC 值，默认为 `false`，表示会生成 CRC，通常仅在错误测试时配置为 `true`。

示例:

```
1| # 为 "lifecycle" 字段选择一个预设值
2| lifecycle = "test"
3|
4| # 为 "uid" 字段提供一个 16 字节的十六进制值
5| uid = "00000000000000000000000000000001"
6|
7| # 为 "bus_cipher_en" 字段选择一个预设值
8| bus_cipher_en = "enable"
9|
10| # 为 "trng_rst_times" 字段提供一个 4 位的二进制值
```


4 密钥文件格式

可参考 `toml-demo/keys/` 目录下的示例文件。

密钥文件用于存储密钥数据，与 `ehsm_imgtool` 使用相同的格式。通过在 `values.toml` 中使用 `key_file` 字段引用密钥文件，可以实现密钥的集中管理和复用。

4.1 支持的密钥类型

密钥文件支持以下密钥类型：

类型	说明	密钥长度
<code>aes128</code>	AES-128 对称密钥	16 字节
<code>aes256</code>	AES-256 对称密钥	32 字节
<code>sm4</code>	SM4 对称密钥（国密）	16 字节
<code>sm2</code>	SM2 非对称密钥（国密）	公钥 65 字节，私钥 32 字节
<code>ecc256</code>	ECC P-256 非对称密钥	公钥 64 字节，私钥 32 字节
<code>rsa2048</code>	RSA-2048 非对称密钥	e: 64 字节，n: 256 字节，d: 256 字节
<code>rsa3072</code>	RSA-3072 非对称密钥	e: 64 字节，n: 384 字节，d: 384 字节

4.2 对称密钥文件格式

对称密钥文件结构简单，包含 `type` 和 `value` 两个字段：

```
1| # SM4 密钥示例
2| type = "sm4"
3| value = "00112233445566778899AABCCDDEEFF"
```

```
1| # AES-256 密钥示例
2| type = "aes256"
3| value = "000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F"
```

4.3 非对称密钥文件格式

非对称密钥文件使用 `[value]` 表来存储公钥和私钥：

SM2/ECC256 密钥：

```
1| type = "sm2"
2|
3| [value]
4| # 公钥（SM2 为 65 字节，含 0x04 前缀；ECC256 为 64 字节）
5| public =
"0428F01618BAC2FA8D69FC5FC4DD18207476215FA3E3B472E024687A73E25F9C3755293EF9E995C748A42BD2B87591502507D90457EB"
```


5 otptool 命令行工具使用

`otptool` 是一个功能完备的命令行工具，包含 `gen`、`parse` 和 `show-layout` 三个子命令。

5.1 `gen` - 生成 OTP 数据

根据 Layout 和 Values 文件生成最终的二进制 OTP 数据和可读的文本描述。

用法:

```
1| otptool gen --layout <LAYOUT_TOML> --values <VALUES_TOML> --output <OUTPUT_BIN> [--  
parse <OUTPUT_TXT>] [--c-array <OUTPUT_C>]  
2| otptool gen --layout <LAYOUT_TOML> --interactive --output <OUTPUT_BIN> [--parse  
<OUTPUT_TXT>] [--c-array <OUTPUT_C>]
```

- `--layout <LAYOUT_TOML>`: 指定 Layout 文件的路径。
- `--values <VALUES_TOML>`: 指定 Values 文件的路径。与 `--interactive` 模式冲突。
- `--output <OUTPUT_BIN>`: 指定生成的二进制 OTP 数据文件的输出路径。
- `--parse <OUTPUT_TXT>`: (可选) 指定生成的可读文本文件的输出路径。
- `--c-array <OUTPUT_C>`: (可选) 指定输出的 C 数组源文件路径 (须以 `.c` 为后缀)。工具会根据文件名自动推导变量名: 去掉 `.c` 后缀并加上 `g_` 前缀。例如指定 `otp_data.c` 时, 生成的变量名为 `g_otp_data`。
- `--interactive, -i`: (可选) 启用交互式模式。在此模式下, 工具将逐个提示用户输入每个字段的值, 而无需 `values.toml` 文件。
- `--watch, -w`: (可选) 启用监控模式。当 `layout.toml` 或 `values.toml` 文件发生变化时, 工具将自动重新生成 OTP 数据。

5.2 `parse` - 解析 OTP 数据

根据 Layout 文件解析一个已有的二进制 OTP 数据文件, 并生成可读的文本描述。

用法:

```
1| otptool parse --layout <LAYOUT_TOML> --input <INPUT_BIN> --parse <OUTPUT_TXT>
```

- `--layout <LAYOUT_TOML>`: 指定用于解析的 Layout 文件路径。
- `--input <INPUT_BIN>`: 指定要解析的二进制 OTP 数据文件路径。
- `--parse <OUTPUT_TXT>`: 指定解析后的可读文本文件的输出路径, 该文件详细描述了每个字段的值。

5.3 `show-layout` - 可视化布局

以表格形式清晰地展示 OTP 内存布局, 帮助开发者快速理解和调试。

用法:

```
1| otptool show-layout --layout <LAYOUT_TOML>
```

- `--layout <LAYOUT_TOML>`: 指定要可视化的 Layout 文件路径。

示例输出:

1	+-----+-----+-----+
2	Field Name Offset Size (bits)
3	+-----+-----+-----+
4	uid 0B + 0b 64
5	security_en 8B + 0b 1
6	(Unused) 8B + 1b 7
7	key_alg_sel 9B + 0b 2
8	(Unused) 9B + 2b 494
9	+-----+-----+-----+

6 C 库使用

`otptool_clib` 提供了 C 语言接口，方便将 OTP 工具集成到其他程序中。

相关文件：

- 静态库: `libotptool_clib.a`
- 头文件: `otptool_clib.h`

我们分别提供了 Windows（基于 mingw）和 Linux 版的 64 位库文件，分别位于 `x86_64-windows` 和 `x86_64-linux` 目录下。

对于接口的使用方式，请参考 `c-src-demo/test.c` 代码。

6.1 `gen_c_array` - 生成 C 数组源文件

将已有的 OTP 二进制数据转换为 C 数组源文件字符串，便于在 C/C++ 项目中直接引用 OTP 数据。

函数签名：

```
1| uint32_t gen_c_array(  
2|     const uint8_t *otp_data,    // 输入: OTP 二进制数据  
3|     uint32_t otp_data_len,      // 输入: 数据长度  
4|     const char *var_name,       // 输入: C 变量名 (如 "g_otp_data")  
5|     char *c_array_buf,         // 输出: C 数组字符串缓冲区  
6|     uint32_t c_array_len,       // 输入: 缓冲区长度  
7|     char *err_msg_buf,         // 输出: 错误信息缓冲区, 可为 NULL  
8|     uint32_t err_msg_len       // 输入: 错误信息缓冲区长度  
9| );
```

返回值: `0` 表示成功，非零表示错误。

输出示例：

```
1| // clang-format off  
2| const unsigned char g_otp_data[24] = {  
3|     /* 0000 */ 0x14, 0x84, 0x81, 0x42, 0x00, 0x00, 0x00, 0x00, ...  
4|     /* 0010 */ 0x00, 0x00, 0x00, 0x01, 0x7f, 0x3f, 0xec, 0x51,  
5| };  
6| // clang-format on
```

7 示例

假设我们有 `layout.toml` 和 `values.toml`。

`layout.toml`:

```
1| [config]
2| otp_size = 64
3| default_one = false
4|
5| [[bytes]]
6| name = "uid"
7| offset = 0
8| length = 8
9|
10| [[bits]]
11| name = "key_alg_sel"
12| offset = 9
13| bit_offset = 0
14| width = 2
15| choices = { aes128 = "10", sm4 = "00" }
16|
17| [[bits]]
18| name = "security_en"
19| offset = 8
20| bit_offset = 0
21| width = 1
22| choices = { "yes" = "1", "no" = "0" }
```

`values.toml`:

```
1| uid = "DEADBEEFCAFEBABE"
2| security_en = "yes"
3| key_alg_sel = "sm4"
```

生成操作:

```
1| optool gen --layout layout.toml --values values.toml --output otp.bin --parse
otp_generated.txt
```

- `otp.bin` 将会包含 `DEADBEEFCAFEBABE` 和 1 在对应位置的二进制数据。
- `otp_generated.txt` 会包含每个字段的详细信息。

生成并同时输出 C 数组源文件:


```
1| otptool gen --layout layout.toml --values values.toml --output otp.bin --c-array  
otp_data.c
```

- `otp_data.c` 将包含一个名为 `g_otp_data` 的 C 数组，可在 C/C++ 项目中直接编译使用。

解析操作:

```
1| otptool parse --layout layout.toml --input otp.bin --parse otp_parsed.txt
```

- `otp_parsed.txt` 的内容应该与 `otp_generated.txt` 的内容一致，用于验证数据正确性。

8 常见问题

- Q: 如果 `values.toml` 中缺少某个字段会怎样?
 - A: 该字段在 OTP 数据中将使用默认值填充, 即由 `layout.toml` 中 `config.default_one` 的设置 (0 或 1) 决定。
- Q: 生成时出现 “field overlaps” 错误是什么意思?
 - A: 这意味着在 `layout.toml` 文件中, 您定义的两个或多个字段在内存空间上存在重叠。请检查每个字段的 `offset`, `bit_offset`, `length`, 和 `width`, 确保它们各自占据独立的空间。
- Q: 按照 eHSM TRM 文档描述的配置值生成的 OTP 数据与预期不符?
 - A: 请检查 `layout.toml` 和 `values.toml` 中的字段定义是否正确。特别注意 `bit_offset` 和 `width` 的设置是否与 TRM 文档一致, 并且注意字节序和位序问题, TRM 文档中位序是大端的, 在 `layout.toml` 和 `values.toml` 中需要转换为小端。
- Q: 在 `values.toml` 的最后写上类似 `uid = "0102030405060708..."` 这样的配置值, 发现生成的 OTP 数据中没有设置对应值?
 - A: 这和 toml 文件语法有关。通常 `values.toml` 的最后是某个密钥值的定义, 如 `[key9] ...`, 这时在后面加上 `uid = "..."` 时, `uid` 这个字段会被解析成 `key9` 的属性 (然后被忽略), 而非全局的配置字段, 导致配置未生效。请将全局字段放在配置文件的前面以避免这种问题。